Комьюнити учеников Яндекс Лицея!
Мероприятия, турниры, конкурсы, новые знакомства — ждём тебя 😎

Подробнее

Урок 6. Как спастись из матрёшки циклов и пошпионить за кодом. Методический материал.

- Общая информация об уроке
- Структура урока
- Блок 1. Переменные-флаги
- ✅ Задача "У меня лапки"
- Блок 2. break и continue
- Блок 3. else у циклов
- ✅ Задача "Тостер"
- ✅ Задача "Не спать!"
- Блок 4. Вложенные циклы
- ✅ Задача "Быть или"
- ✅ Задача "Температурный рекорд"
- Блок 5. Выход из вложенных циклов
- ✅ Задача "Сумма нечётных"
- Блок 6. Отладка программ
- ✅ Задача "Везде жуки"
- Завершение урока
- Таблица задач урока

Отступать от рекомендаций — нормально, главное, чтобы ученики усвоили необходимые понятия и решили задачи.

Общая информация об уроке

Тип урока: Новая тема
Предварительные знания учеников: Переменные, типы данных, арифметические операции, преобразование типов, ввод и вывод данных, f-строки, условный оператор `if-elif-else`, цикл `while`, цикл `for`, `range()`, `break`, базовые операции со строками (уроки 1–5)

На этом уроке ученики научатся:

- Использовать булевы переменные как флаги для управления ходом программы.
- Применять оператор `break` для немедленного выхода из цикла.
- Применять оператор `continue` для пропуска текущей итерации.
- Использовать блок `else` у циклов `for` и `while`.
- Писать вложенные циклы и понимать порядок их выполнения.
- Выходить из нескольких уровней вложенности с помощью флага и `break`.
- Применять базовые приёмы отладки: отладочный `print()` и дебаггер IDE.

Структура урока

№	Блок	Ориентировочное время	Задачи
1	Переменные-флаги	7–9 мин	—
2	**Задача "У меня лапки"**	5–8 мин	Флаг не нужен, `for` + условие с `not` и `or`
3	Операторы `break` и `continue`	7–9 мин	—
4	Блок `else` у циклов	5–8 мин	—
5	**Задача "Тостер"**	6–8 мин	`break` в `for`, накопитель, форматированный вывод
6	**Задача "Не спать!"**	7–9 мин	`while` + `break`, `len()`, арифметика остатка
7	Вложенные циклы	7–9 мин	—
			`while` по пустой строке, `for` по

№	Блок	Ориентировочное время	Задачи
9	**Задача "Температурный рекорд"**	7–9 мин	Вложенные `while`/`for` + `while`, накопление максимума
10	Выход из вложенных циклов (флаг + `break`)	6–8 мин	—
11	**Задача "Сумма нечётных"**	7–9 мин	`while` + `else`, преобразование `int → str → int`
12	Отладка программ	7–9 мин	—
13	**Задача "Везде жуки"**	7–9 мин	`while` + `break`, проверка подстроки и отдельных символов
14	Подведение итогов	2–3 мин	—

🧠 **Темп урока:** Урок насыщен механиками управления циклами — не торопитесь с `break`/`continue`: ученики путают их друг с другом. Уделите время сравнению «`флаг vs break`» — оба подхода правильные, важно понять, где что уместнее. Блок про вложенные циклы + выход из них лучше разбирать на конкретном примере с рисунком на доске. Отладка — завершающий блок, не обрезайте его: это навык на всю жизнь.

Блок 1. Переменные-флаги

Время: 7–9 минут. Концептуальный блок — не пропускайте, задачи урока активно используют этот паттерн.

Ключевые идеи для объяснения

- Флаг — это булева переменная, которая «запоминает», произошло ли некоторое событие в ходе работы программы. Изначально `False`, а при наступлении события становится `True`.
- Чаще всего применяется с `while`, где флаг — либо единственное условие выхода, либо дополнительный «аварийный выход»:

```
ready_to_exit = False
while not ready_to_exit:
```

```
    if command == 'exit':
        ready_to_exit = True
```

- Флаг может применяться и с `for`, хотя сам по себе он не остановит цикл досрочно — мы просто фиксируем факт события, а реакция на него будет уже после цикла.
- Соглашение по именованию: имя флага должно звучать как утверждение — `need_exit`, `has_cat`, `was_pressed`. Не `flag1`, не `b`.

🧠 Ученики иногда пытаются использовать несколько флагов там, где хватает одного. Если флагов в коде становится много — это обычно признак того, что логику можно упростить.

✅ Задача "У меня лапки" (10 баллов)

Вводить после объяснения переменных-флагов. У учеников достаточно знаний для решения.

```
check_number_1 = int(input())
check_number_2 = int(input())
start = int(input())
stop = int(input())
for i in range(start, stop + 1):
    if not (i % check_number_1 == 0 or i % check_number_2 == 0):
        print(i, end=' ')
```

Что проверяется: `range(start, stop + 1)` с включительной верхней границей; логическое выражение с `not` и `or`; вывод через `end=' '`.

Как вести на уроке:
Разберите условие вслух: «вывести числа, которые НЕ делятся ни на `check_number_1`, ни на `check_number_2`». Ключевой вопрос: как записать «не делится ни на одно из двух»? `not (делится на первое OR делится на второе)` — закон де Моргана (не обязательно называть его так). Альтернатива: `i % check_number_1 != 0 and i % check_number_2 != 0`. Про флаги: задача решается без флага, но использует `for` с условием — хороший момент обсудить, нужен ли флаг вообще.

Блок 2. Операторы `break` и `continue`

Время: 7–9 минут. Центральный блок — оба оператора будут активно использоваться в

- `break` — немедленный выход из цикла, независимо от того, закончились ли итерации:

```
for num in 1, 3, 5, 8, 9, 7:
    if num % 2 == 0:
        print(f"Первое чётное: {num}")
        break
```

- `continue` — пропустить оставшуюся часть текущей итерации и перейти к следующей:

```
for i in range(6):
    if i == 2:
        continue
    print(i)  # 0, 1, 3, 4, 5
```

- **Осторожность с `continue` в `while`:** если переменная-счётчик меняется в конце тела цикла, `continue` может «перепрыгнуть» через её изменение и создать бесконечный цикл.
- Сравнение «флаг vs `break`»: оба работают, но `break` компактнее; флаг лучше там, где нужно ещё и знать, что именно произошло.

🧠 `break` без `if` сделает цикл бессмысленным: первая же итерация — и сразу выход. Если ученик пишет `break` не внутри условия, скорее всего, это ошибка.

Блок 3. Блок `else` у циклов

Время: 5–8 минут.

- У циклов `for` и `while` есть необязательный блок `else`. Он выполняется, если цикл завершился **без `break`**.
- Если `break` сработал — блок `else` пропускается.
- Классический пример — поиск с проверкой «нашли / не нашли»:

```
password = "secret"
attempts = 3
while attempts > 0:
    guess = input("Введите пароль: ")
    if guess == password:
        print("Доступ разрешён!")
        break
```

```
else:
    print("Доступ запрещён.")
```

🧠 Блок `else` у цикла — нечастая конструкция, и многие опытные программисты её избегают из-за неочевидности. Но в контексте задач курса она встречается — объясните её смысл максимально конкретно.

✅ Задача "Тостер" (15 баллов)

Вводить после объяснения `break`. Можно на её примере разобрать `else` у `for`.

```
result = int(input())
t = int(input())
stepen = 0
for i in range(t):
    power = int(input())
    stepen += power
    if stepen >= result:
        print(f'Ваш хлеб уже готов! Степень поджаривания {stepen}.')
        break
else:
    print(f'Хлеб поджарен до степени {stepen}.')
```

Что проверяется: `break` для выхода из `for` при достижении условия; накопление суммы; `else` у цикла (если цикл завершился без `break`).

Как вести на уроке:
Обратите внимание: здесь уже используется `else` у цикла — хороший мостик к следующему теоретическому блоку. Разберите логику: если сумма достигла нужной степени — хлеб готов (выходим с `break`), если все итерации прошли, но не достигли — выводим итоговую степень (блок `else`).

✅ Задача "Не спать!" (15 баллов)

Вводить после объяснения `break` и `continue`.

```
power_line = input()
thing_line = input()
power = len(power_line)
while thing_line != 'Можно спать.':
```

```
    elif len(thing_line) % 2 == 0:
        power -= len(thing_line) // 2
    else:
        power -= 1
    if power <= 0:
        print(thing_line)
        break
    thing_line = input()
else:
    print(power)
```

Что проверяется: `while` с условием по строке-сигналу; `break` при критическом состоянии; `else` у `while`; приоритет проверки делимости (сначала на 3, затем на 2); `len()` для вычислений.

Как вести на уроке:
Разберите задачу по слоям: откуда берётся `power` (длина первой строки), как он уменьшается (зависит от длины `thing_line`), когда цикл заканчивается (строка-сигнал или `power ≤ 0`). Подчеркните порядок `elif`: если `len % 3 == 0` — первый приоритет, иначе проверяем `% 2`.

Блок 4. Вложенные циклы

Время: 7–9 минут.

- В Python цикл можно поместить внутрь другого цикла. Внешний цикл — с меньшим уровнем вложенности, внутренний — с большим.
- **Ключевое правило:** каждую итерацию внешнего цикла внутренний цикл выполняется **полностью**.
- Пример — таблица умножения:

```
k = int(input())
for j in range(1, k + 1):
    for i in range(1, 10):
        print(f"{i} * {j} = {j * i}", end='\t')
    print()
```

- Сложность вложенных циклов: если оба цикла выполняются n раз — итого n^2 итераций.

`for i in range(m)`
во внутреннем с одним и тем же именем переменной. Напомните: имена переменных вложенных циклов
должны различаться
, иначе переменная внешнего цикла будет затёрта.

✅ Задача "Быть или" (15 баллов)

Вводить после объяснения `break` и `continue`. У учеников достаточно знаний для решения.

```
line = input()
while line != '':
    for symbol in line:
        if symbol not in 'aeyuioAEYUIO':
            print(symbol, end='')
    print()
    line = input()
```

Что проверяется: `while` по пустой строке как сигналу завершения; вложенный `for` по символам строки; фильтрация символов через `not in`; `print()` без аргументов для перевода строки.

Как вести на уроке:
Обратите внимание учеников на идиому `while line != ''`: цикл читает строки до тех пор, пока не встретит пустую. Ключевой вопрос: зачем `print()` после внутреннего цикла? Чтобы перейти на новую строку после обработки каждой строки ввода.

✅ Задача "Температурный рекорд" (15 баллов)

Вводить после объяснения `else` у цикла.

```
days = int(input())
max_temperature = -1000
day = 0
for i in range(days):
    t = input()
    while t != 'New day':
        if float(t) > max_temperature:
            max_temperature = float(t)
```

```
        t = input()
print(max_temperature, day)
```

Что проверяется: вложенные цикл `for` (по дням) и `while` (по показаниям внутри дня); накопление максимума с запоминанием номера дня; `float()` для нецелых температур; сигнальная строка `'New day'` для разделения дней.

Как вести на уроке:
Разберите структуру ввода: сначала количество дней, потом для каждого дня — строки с температурами, заканчивающиеся строкой `'New day'`. Ключевой вопрос: зачем инициализировать `max_temperature = -1000`? Чтобы любая реальная температура заведомо оказалась больше начального значения.

Блок 5. Выход из вложенных циклов

Время: 6–8 минут.

- `break` и `continue` работают **только в том цикле, в котором непосредственно стоят**. Нельзя одним `break` выйти из двух вложенных циклов.
- Способ выхода из нескольких уровней — комбинация флага и `break`:

```
runaway = False
for j in range(1, 100):
    for i in range(1, 100):
        if i * j > 700:
            runaway = True
            break
    if runaway:
        break
```

✅ Задача "Сумма нечётных" (15 баллов)

Вводить после объяснения `else` у цикла.

```
number = int(input())
while number != 17:
    number = str(number)
    s = 0
    for digit in number:
        if int(digit) % 2 != 0:
            print(s)
```

```
    else:
        print('Ну, я так не играю!')
        break
    number = int(input())
else:
    print('Какие замечательные числа!')
```

Что проверяется: `while` + `else`; `break` при особом условии; вложенный `for` по цифрам числа (через преобразование в строку).

Как вести на уроке:
Покажите трюк: `number = str(number)` превращает число в строку, чтобы перебирать цифры поштучно. Разберите три исхода: встретили 17 — выходим через `while`'s `else`; сумма нечётных цифр равна нулю — `break`; иначе — выводим сумму, продолжаем.

Блок 6. Отладка программ

Время: 7–9 минут.

- **Метод 1 — отладочный `print()`:** вставить вывод переменных в «подозрительные» места. Плюс: быстро и просто. Минус: нет гарантии, что смотрите в правильное место. Удалите отладочные `print()` перед сдачей.
- **Метод 2 — дебаггер IDE:** расставить точки останова (*breakpoints*), запустить дебаггер, шагать по строкам клавишей F10, наблюдать за значениями переменных.
- Алгоритм отладки: поставить breakpoint в «подозрительном» месте → запустить дебаггер → шагать по коду → найти момент, когда переменная принимает неожиданное значение → понять причину → исправить.

🧠 Подчеркните важность отладки как профессионального навыка. По статистике, разработчики тратят больше времени на поиск ошибок, чем на написание нового кода. Умение пользоваться дебаггером — конкурентное преимущество.

✅ Задача "Везде жуки" (15 баллов)

Вводить после объяснения вложенных циклов и выхода из них. Можно использовать для иллюстрации отладки.

```
line = input()
```

```
        print(line)
        break
    elif 'b' in line and 'u' in line and 'g' in line or 'B' in line and '
        print('BUG')
    else:
        print()
    line = input()
```

Что проверяется: `while` + `break`; проверка подстроки через `in`; проверка отдельных букв через несколько `and`; `print()` для вывода пустой строки; приоритет операторов `and`/`or`.

Как вести на уроке:

Разберите три состояния входной строки: содержит подстроку `bug` или `BUG` — выводим строку и `break`; содержит все три буквы по отдельности — выводим `'BUG'`; иначе — пустая строка. Акцент на приоритете: `and` выполняется раньше `or`.

Завершение урока

Время: 2–3 минуты.

- Сегодня вы получили полный арсенал для работы с циклами: `break` выходит из цикла, `continue` пропускает итерацию, `else` выполняется, если `break` не сработал. Флаги помогают передавать информацию между уровнями вложенности.
- Напомнить про **классные и домашние задачи**: если что-то не успели — продолжить дома.
- **Домашнее задание:** дочитать урок в учебнике, выполнить домашние задачи в ЛМС.
- Напомнить, где задавать вопросы: чат задачи в ЛМС и групповой чат.

Таблица задач урока

Классная работа

Задача	Баллы	Что проверяется	На что обратить особое внимание
У меня гатки	10	`range(start, stop + 1)`; логическое выражение `not`	Альтернатива через `and` с `!=`. Граница диапазона

Задача	Баллы	Что проверяется	На что обратить особое внимание
Тостер	15	`break` при достижении суммы; накопитель `stepen`; `else` у `for`	`else` относится к `for`, не к `if`. Граничный случай `t = 0` — сработает только `else`
Не спать!	15	`while` + `break`; приоритет `% 3` над `% 2`; `else` у `while`; `len()`	`thing_line = input()` стоит в конце тела, не в начале. `continue` здесь не нужен
Быть или	15	`while` по пустой строке; вложенный `for` по символам; `not in` для гласных; `print()` для перевода строки	Список гласных должен включать и строчные, и заглавные. `print()` без аргументов после `for`
Температурный рекорд	15	Вложенные `for` + `while`; накопление максимума с номером дня; `float()`; сигнальная строка	`day = i + 1` (индексация с 1). Проверка `!= 'New day'` до `float()`
Сумма нечётных	15	`while` + `else`; `break` при нулевой сумме; перебор цифр через `str()`	`else` у `while` — не у `if`. Трюк с `number = str(number)` для перебора цифр
Везде жуки	15	`while` + `break`; проверка подстроки `in`; проверка отдельных букв; приоритет `and`/`or`	Три отдельных условия с правильным порядком приоритетов. `print()` для пустой строки

Домашние задачи

Задача	Баллы	Что проверяется	На что обратить особое внимание
Календарь	25	`range(1, days + 1, 7)` с шагом 7; условие неполной недели; `print(*range(...), sep='\t')`	Граничное условие `i + 6 <= days` для полной недели. Оператор `*` для распаковки `range`
Сине-зелёные	25	`continue` для пропуска "красных" строк; независимые	`from math import ceil`. Если `num_blue == 0` —

Задача	Баллы	Что проверяется	На что обратить особое внимание
			может содержать и `green`, и `blue` одновременно
Сингулярность	25	`while` с условием по сумме координат; `dist()` из `math`; `round(..., 4)`; накопление суммы расстояний; `break` при попадании в радиус	`from math import dist`. Условие `while x + y <= 10_000`. `else` у `while` — для вывода суммы, если `break` не сработал
Телепортатор	25	Перебор цифр строки в цикле `for`; перевод из произвольной системы счисления в десятичную; `else` у `for`; проверка недопустимых цифр (`int(digit) >= base`)	`else` срабатывает только если все цифры допустимы. Алгоритм перевода: `number_10 = number_10 * base + int(digit)` — разобрать пошагово